



Python Programming Fundamentals

List Comprehensions

Programming Fundamentals Team

SETU

2026-06-12



Outline

- 1. What is a List Comprehension?
- 2. Syntax
- 3. Filtering with if
- 4. List Comprehensions with Objects
- 5. Dict Comprehensions
- 6. When to Use Comprehensions vs Loops
- 7. Nested Comprehensions



Outline

- 1. What is a List Comprehension?
- 2. Syntax
- 3. Filtering with if
- 4. List Comprehensions with Objects
- 5. Dict Comprehensions
- 6. When to Use Comprehensions vs Loops
- 7. Nested Comprehensions



1.1 Concise list construction

A list comprehension builds a new list from an existing iterable in **one line**.

Traditional loop:

```
squares = []  
for n in range(1, 6):  
    squares.append(n ** 2)  
# [1, 4, 9, 16, 25]
```

List comprehension:

```
squares = [n ** 2 for n in range(1, 6)]  
# [1, 4, 9, 16, 25]
```



Outline

- 1. What is a List Comprehension?
- 2. Syntax**
- 3. Filtering with if
- 4. List Comprehensions with Objects
- 5. Dict Comprehensions
- 6. When to Use Comprehensions vs Loops
- 7. Nested Comprehensions

2.1 [expression for item in iterable]

[expression	for	variable	in	iterable]
^^^^^^^^^^		^^^^^^^^		^^^^^^^^
what to put		loop var		source
in the list				

Examples:

```
names = ["apple", "bread", "milk"]
```

```
# Uppercase all names
upper = [n.upper() for n in names]
# ['APPLE', 'BREAD', 'MILK']
```

```
# Get lengths
lengths = [len(n) for n in names]
# [5, 5, 4]
```



Outline

- 1. What is a List Comprehension?
- 2. Syntax
- 3. Filtering with if**
- 4. List Comprehensions with Objects
- 5. Dict Comprehensions
- 6. When to Use Comprehensions vs Loops
- 7. Nested Comprehensions



3.1 [expression for item in iterable if condition]

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
evens = [n for n in numbers if n % 2 == 0]
# [2, 4, 6, 8, 10]
```

```
odds = [n for n in numbers if n % 2 != 0]
# [1, 3, 5, 7, 9]
```

With objects:

```
products = shop.get_all_products()
```

```
cheap = [p for p in products if p.get_price() < 2.00]
expensive = [p for p in products if p.get_price() >= 3.00]
names_only = [p.get_name() for p in products]
```




Outline

- 1. What is a List Comprehension?
- 2. Syntax
- 3. Filtering with if
- 4. List Comprehensions with Objects**
- 5. Dict Comprehensions
- 6. When to Use Comprehensions vs Loops
- 7. Nested Comprehensions



4. List Comprehensions with Objects

```

from shop import Shop
from product import Product

shop = Shop("Test")
shop.add_product(Product("Apple", 1.99, 101))
shop.add_product(Product("Bread", 2.50, 102))
shop.add_product(Product("Milk", 1.20, 103))
shop.add_product(Product("Cheese", 4.50, 104))

products = shop.get_all_products()

# Names of products costing less than €2
cheap_names = [p.get_name()
                for p in products
                if p.get_price() < 2.00]
# ['Apple', 'Milk']
    
```



4. List Comprehensions with Objects

```
# Prices as a list
prices = [p.get_price() for p in products]
# [1.99, 2.50, 1.20, 4.50]
```



Outline

- 1. What is a List Comprehension?
- 2. Syntax
- 3. Filtering with if
- 4. List Comprehensions with Objects
- 5. Dict Comprehensions**
- 6. When to Use Comprehensions vs Loops
- 7. Nested Comprehensions

5. Dict Comprehensions

```
products = shop.get_all_products()

# {name: price} dictionary
price_map = {p.get_name(): p.get_price()
              for p in products}
# {'Apple': 1.99, 'Bread': 2.50, ...}

# {id: product object}
id_map = {p.get_product_id(): p
          for p in products}
# {101: <Product>, 102: <Product>, ...}

# Look up a product by id
product_101 = id_map.get(101)
```



Outline

- 1. What is a List Comprehension?
- 2. Syntax
- 3. Filtering with if
- 4. List Comprehensions with Objects
- 5. Dict Comprehensions
- 6. When to Use Comprehensions vs Loops**
- 7. Nested Comprehensions

6. When to Use Comprehensions vs Loops

Use comprehension	Use a regular loop
Simple transformation	Complex multi-step logic
Filtering a list	Multiple statements per iteration
Building a new list quickly	Side effects (printing, etc.)
One-line readable expression	When clarity suffers

Rule of thumb: if it fits on one readable line, a comprehension is fine.



Outline

- 1. What is a List Comprehension?
- 2. Syntax
- 3. Filtering with if
- 4. List Comprehensions with Objects
- 5. Dict Comprehensions
- 6. When to Use Comprehensions vs Loops
- 7. Nested Comprehensions**



7. Nested Comprehensions

```
# Flatten a 2D list
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flat = [n for row in matrix for n in row]
# [1, 2, 3, 4, 5, 6, 7, 8, 9]

# All combinations
colours = ["red", "blue"]
sizes = ["S", "M", "L"]
combos = [(c, s) for c in colours for s in sizes]
# [('red', 'S'), ('red', 'M'), ...]
```

Use nested comprehensions sparingly — readability matters.



Thanks for Watching - Any questions?

